



Christophe PICHAUD
Architecte Microsoft chez
'Modern Applications by Devoteam'
christophepichaud@hotmail.com
www.windowscpp.com

C++



XAML Islands : incorporez les contrôles XAML Windows 10 dans vos Apps WinForms et WPF.

L'interface Windows 10 WinRT XAML ou UWP est faite en C++. Microsoft souhaite que les développeurs créent des applications XAML et les mettent à disposition sur le store. Il en va de même pour les applications WinForms ou WPF. Pour cela, l'éditeur propose XAML Islands, un SDK qui permet d'intégrer des composants WinRT XAML dans les applications WinForms et WPF. Ceci est valable depuis Windows 10 version 1903.

niveau
200

Comment ça marche

Il y a deux façons de faire, à savoir pour :

- Les applications C++ Win32 ;
- Les applications WinForms, WPF NET Framework et Net Core.

Windows 10 version 1903

À partir de Windows 10, version 1903, Microsoft fournit deux façons d'utiliser XAML Islands dans vos applications WPF, Windows Forms et C++ Win32 :

- Le kit de développement logiciel (SDK) Windows fournit plusieurs classes Windows Runtime et interfaces COM que votre application peut utiliser pour héberger tout contrôle UWP qui dérive de **Windows.UI.Xaml.UIElement**. Collectivement, ces classes et interfaces sont appelées l'API d'hébergement XAML UWP, et elles vous permettent d'héberger des contrôles UWP dans n'importe quel élément d'interface utilisateur dans votre application qui a un handle de fenêtre associé (HWND). Pour plus d'informations sur cette API, consultez Utilisation de l'API d'hébergement XAML.
- **Windows Community Toolkit** fournit également des contrôles XAML Islands supplémentaires pour WPF et Windows Forms. Ces contrôles utilisent l'API d'hébergement XAML UWP en interne et implémentent tout le comportement que vous auriez autrement besoin de gérer vous-même si vous avez utilisé l'API d'hébergement XAML UWP directement, y compris la navigation au clavier et les modifications de disposition. Pour les applications WPF et Windows Forms, nous vous recommandons vivement d'utiliser ces contrôles à la place de l'API d'hébergement XAML UWP directement, car ils implémentent beaucoup de détails d'implémentation de l'utilisation de l'API. Notez qu'à partir de Windows 10, version 1903, ces contrôles sont disponibles en tant que preview pour les développeurs.

Les applications de bureau C++ Win32 doivent utiliser l'API d'hébergement XAML UWP pour héberger des contrôles UWP. Les contrôles XAML Islands dans le Windows Community Toolkit ne sont pas disponibles pour les applications de bureau C++ Win32. Il existe deux types de contrôle XAML Islands dans le Windows Community Toolkit pour les applications WPF et WinForms : contrôles encapsulés et contrôles hôtes.

Contrôles encapsulés

Les applications WPF et Windows Forms peuvent utiliser une sélection de contrôles UWP encapsulés dans le Windows Community Toolkit. Ils sont appelés contrôles encapsulés, car ils encapsulent l'interface et les fonctionnalités d'un contrôle UWP spécifique. Vous pouvez ajouter ces contrôles directement à l'aire de conception de votre projet WPF ou Windows Forms, puis les utiliser comme tout autre contrôle WPF ou Windows Forms dans votre designer.

Les contrôles UWP encapsulés suivants pour implémenter XAML Islands sont actuellement disponibles pour les applications WPF et Windows Forms.

Control	Minimum supported OS	Description
InkCanvas InkToolbar	Windows 10, version 1903	Fournit une surface et des barres d'outils associées pour l'interaction utilisateur basée sur Windows Ink dans votre application de bureau Windows Forms ou WPF.
MediaPlayerElement	Windows 10, version 1903	Incorpore une vue qui diffuse et restitue le contenu multimédia tel que la vidéo dans votre application de bureau Windows Forms ou WPF.
MapControl	Windows 10, version 1903	Vous permet d'afficher une carte symbolique ou photoréaliste dans votre application de bureau Windows Forms ou WPF.

En plus des contrôles encapsulés pour les îles XAML, Windows Community Toolkit fournit également les contrôles suivants pour l'hébergement de contenu Web.

Control	Minimum supported OS	Description
WebView	Windows 10, version 1803	Utilise le moteur de rendu Microsoft Edge pour afficher le contenu Web.
WebViewCompatible	Windows 7	Fournit une version de WebView qui est compatible avec d'autres versions du système d'exploitation. Ce contrôle utilise le moteur de rendu Microsoft Edge pour afficher le contenu Web sur Windows 10 version 1803 et versions ultérieures, et le moteur de rendu Internet Explorer pour afficher le contenu Web sur les versions antérieures de Windows 10, Windows 8.x et Windows 7.

Note

Cet article ne couvre pas en détails les applications C++ Win32. Le focus est fait pour les applications .NET.

Hosting de Controls

Pour les scénarios au-delà de ceux couverts par les contrôles encapsulés disponibles comme WebView et Ink Control, les applications WPF et Windows Forms peuvent également utiliser le contrôle WindowsXamlHost du Windows Community Toolkit (<https://docs.microsoft.com/windows/uwpcommunitytoolkit/>). Ce contrôle peut héberger n'importe quel contrôle UWP qui dérive de Windows. UI. Xaml. UIElement, y compris tout contrôle UWP fourni par le SDK Windows ainsi que les contrôles utilisateur personnalisés. Ce contrôle prend en charge Windows 10 Insider Preview SDK Build 17709 et versions ultérieures.

Vue d'ensemble de l'architecture

Voici un aperçu de la façon dont ces contrôles sont organisés architecturalement. **1**

Les API qui apparaissent au bas de ce schéma sont expédiées avec le kit de développement logiciel (SDK) Windows. Les contrôles encapsulés et les contrôles hôtes sont disponibles via les packages NuGet dans le Windows Community Toolkit.

Prérequis

XAML Islands requiert Windows 1903.

Les samples Microsoft du WCT

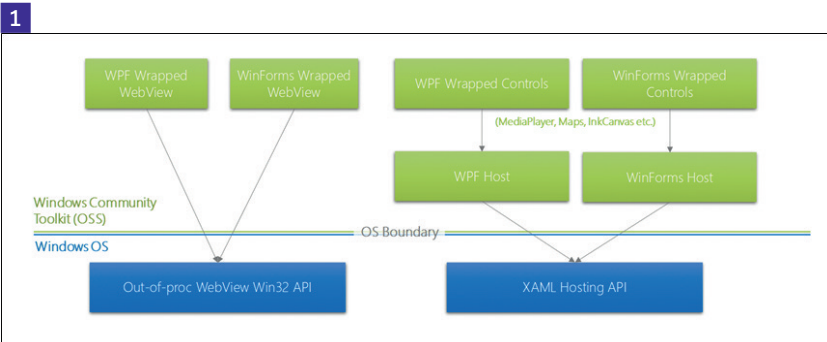
Le Windows Community Toolkit contient des exemples Webview, Ink pour WinForms et WPF et aussi des exemples custom XAML controls pour WinForms et WPF.

Exemple d'application WPF :

```
using System;
using System.Windows;
using Microsoft.Toolkit.Win32.UI.Controls.Interop.WinRT;
using windows = Windows;

namespace Microsoft.Toolkit.Sample.Wpf.App
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    public partial class MainWindow : Window
    {
        private windows.UI.Xaml.Controls.ContentDialog _contentDialog;

        public MainWindow()
    }
}
```



```
{
    InitializeComponent();
}

private void inkCanvas_Loaded(object sender, RoutedEventArgs e)
{
    inkCanvas.InkPresenter.InputDeviceTypes =
        CoreInputDeviceTypes.Mouse |
        CoreInputDeviceTypes.Pen |
        CoreInputDeviceTypes.Touch;
}

private void inkToolbar_Initialized(object sender, EventArgs e)
{
    // Handle ink toolbar initialization events here
}

private void inkToolbar_ActiveToolChanged(object sender, object e)
{
    // Handle ink toolbar active tool changed events here.
}

private async void myMap_Loaded(object sender, RoutedEventArgs e)
{
    // Specify a known location.
    BasicGeoposition cityPosition =
        new BasicGeoposition() { Latitude = 47.604, Longitude = -122.329 };
    var cityCenter = new Geopoint(cityPosition);

    // Set the map location.
    await myMap.TrySetViewAsync(cityCenter, 12).ConfigureAwait(false);
}

private void WindowsXamlHost_Loaded(object sender, RoutedEventArgs e)
{
    windows.UI.Xaml.Controls.StackPanel stackPanel = new windows.UI.Xaml.Controls.
    StackPanel()
    {
        Background = new windows.UI.Xaml.Media.SolidColorBrush(windows.UI.Colors.Black),
    };

    stackPanel.Children.Add(new windows.UI.Xaml.Shapes.Rectangle()
    {
        Width = 50,
        Height = 75,
        Fill = new windows.UI.Xaml.Media.SolidColorBrush(windows.UI.Colors.Blue),
    });

    stackPanel.Children.Add(new windows.UI.Xaml.Shapes.Rectangle()
    {
        Width = 200,
        Height = 30,
        Fill = new windows.UI.Xaml.Media.SolidColorBrush(windows.UI.Colors.Red),
    });

    var button = new windows.UI.Xaml.Controls.Button()
}
```

```

{
    Width = 160,
    Height = 60,
    HorizontalAlignment = windows.UI.Xaml.HorizontalAlignment.Center,
    Content = "ContentDialog UWP Button",
};
button.Tapped += Button_Tapped;
stackPanel.Children.Add(button);

stackPanel.Children.Add(new windows.UI.Xaml.Shapes.Rectangle()
{
    Width = 25,
    Height = 100,
    Fill = new windows.UI.Xaml.Media.SolidColorBrush(windows.UI.Colors.Green),
});

windows.UI.Xaml.Controls.Flyout flyout = new windows.UI.Xaml.Controls.Flyout();
flyout.Content = new windows.UI.Xaml.Controls.TextBlock()
{ Text = "Flyout content", };

var button2 = new windows.UI.Xaml.Controls.Button()
{
    Width = 300,
    Height = 40,
    HorizontalAlignment = windows.UI.Xaml.HorizontalAlignment.Center,
    Content = "Long UWP Button with Flyout",
    Flyout = flyout,
};
stackPanel.Children.Add(button2);

var comboBox = new windows.UI.Xaml.Controls.ComboBox()
{
    HorizontalAlignment = windows.UI.Xaml.HorizontalAlignment.Center,
};
comboBox.Items.Add("One");
comboBox.Items.Add("Two");
comboBox.Items.Add("Three");
comboBox.Items.Add("Four");
stackPanel.Children.Add(comboBox);

windows.UI.Xaml.Controls.Grid grid = new windows.UI.Xaml.Controls.Grid();
stackPanel.Children.Add(grid);

_contentDialog = new windows.UI.Xaml.Controls.ContentDialog();
_contentDialog.Content = new windows.UI.Xaml.Controls.TextBlock()
{ Text = "ContentDialog content", };
stackPanel.Children.Add(_contentDialog);

var popup = new windows.UI.Xaml.Controls.Primitives.Popup()
{
    Width = 50,
    Height = 50,
    ShouldConstrainToRootBounds = false,
    Child = new windows.UI.Xaml.Controls.TextBlock()
    { Text = "Popup child", },
};

```

```

grid.Children.Add(popup);

windowsXamlHost.Child = stackPanel;
popup.IsOpen = true;
}

private async void Button_Tapped(object sender, windows.UI.Xaml.Input.TappedRoutedEventArgs e)
{
    await _contentDialog.ShowAsync(windows.UI.Xaml.Controls.ContentDialogPlacement.Popup);
}
}

```

Voici le contenu de MainWindow.xaml :

```

<Window x:Class="Microsoft.Toolkit.Sample.Wpf.App.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:xamlhost="clr-namespace:Microsoft.Toolkit.Wpf.UI.XamlHost;assembly=Microsoft.Toolkit.Wpf.UI.XamlHost"
    xmlns:controls="clr-namespace:Microsoft.Toolkit.Wpf.UI.Controls;assembly=Microsoft.Toolkit.Wpf.UI.Controls"
    xmlns:controls1="clr-namespace:Microsoft.Toolkit.Wpf.UI.Controls;assembly=Microsoft.Toolkit.Wpf.UI.Controls"
    xmlns:controls2="clr-namespace:Microsoft.Toolkit.Wpf.UI.Controls;assembly=Microsoft.Toolkit.Wpf.UI.Controls"
    xmlns:x1="clr-namespace:System.Windows.Markup;assembly=System.Xaml"
    mc:Ignorable="d"
    Title="MainWindow" Height="450" Width="800">
<Grid>
<TabControl>
<TabItem>
<TabItem.Header>Ink Controls (Canvas / Toolbar)</TabItem.Header>
<DockPanel LastChildFill="True">
<controls:InkToolbar DockPanel.Dock="Top" x:Name="inkToolbar"
    Grid.Row="0" TargetInkCanvas="{x1:Reference Name=inkCanvas}"
    Initialized="inkToolbar_Initialized"
    ActiveToolChanged="inkToolbar_ActiveToolChanged">
<controls:InkToolbarCustomToolButton x:Name="toolButtonLasso" />
</controls:InkToolbar>
<!--Inking area -->
<controls:InkCanvas x:Name="inkCanvas" DockPanel.Dock="Top" Loaded="inkCanvas_Loaded"/>
</DockPanel>
</TabItem>
<TabItem>
<TabItem.Header>Samples</TabItem.Header>
<xamlhost:WindowsXamlHost x:Name="windowsXamlHost" Loaded="WindowsXamlHost_Loaded" />
</TabItem>
<TabItem>
<TabItem.Header>MediaPlayerElement</TabItem.Header>
<controls:MediaPlayerElement x:Name="mediaPlayerElement"
    Source="https://mediaplatstorage1.blob.core.windows.net/windows-universal

```

```
-samples-media/elephantsdream-clip-h264_sd-aac_eng-aac_spa-aac_eng_commentary-srt_eng-srt_por-srt_swe.mkv"
```

```
AutoPlay="True" Margin="5" HorizontalAlignment="Stretch"
VerticalAlignment="Stretch" AreTransportControlsEnabled="True" />
```

```
</TabItem>
```

```
<TabItem>
```

```
<TabItem.Header>WebView</TabItem.Header>
```

```
<!--WebBrowser /-->
```

```
<controls:WebViewCompatible Source="http://www.bing.com"
HorizontalAlignment="Stretch" VerticalAlignment="Stretch"/>
```

```
<!--WebBrowser Source="http://www.bing.com"/-->
```

```
</TabItem>
```

```
<TabItem>
```

```
<TabItem.Header>MapControl</TabItem.Header>
```

```
<controls:MapControl x:Name="myMap" Loaded="myMap_Loaded"/>
```

```
</TabItem>
```

```
</TabControl>
```

```
</Grid>
```

```
</Window>
```

Voici ce que cela donne dans Visual Studio 2019 : 2

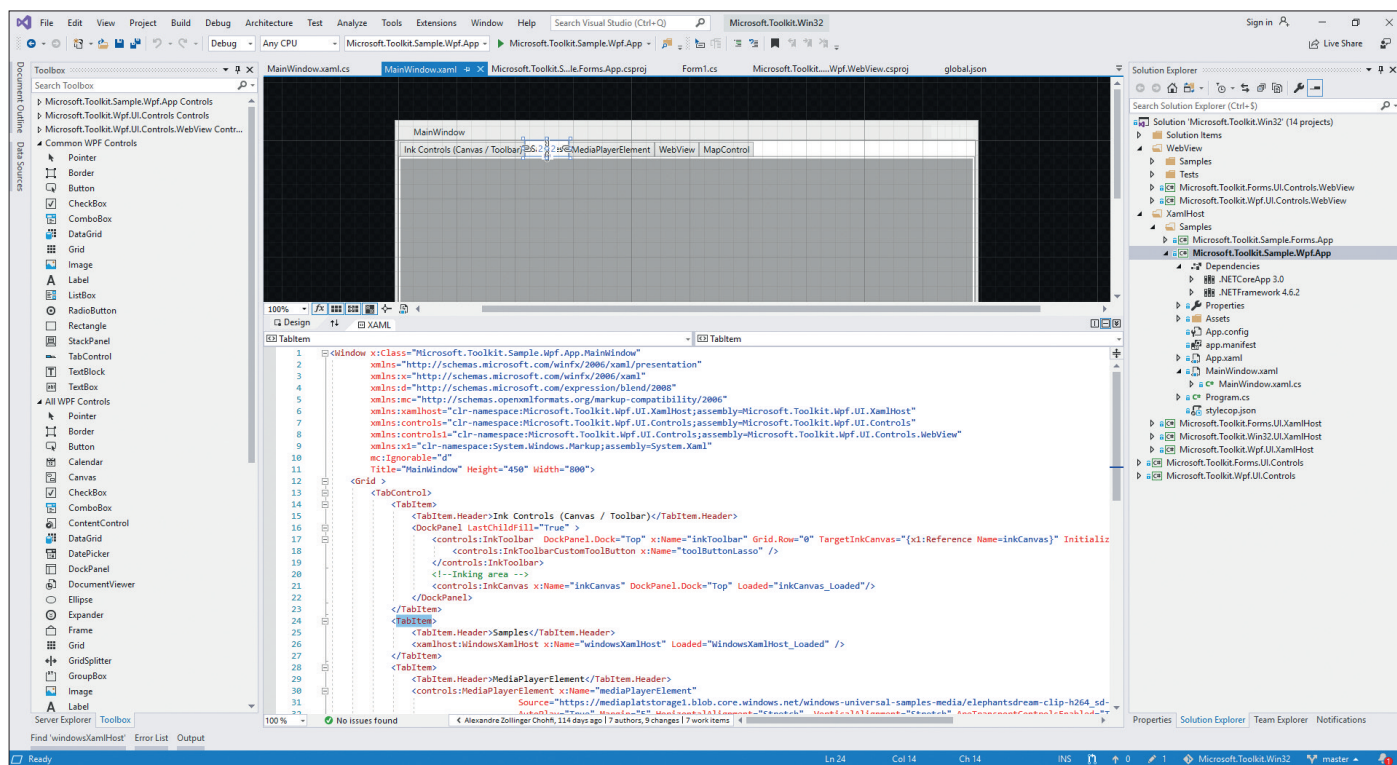
Conclusion

L'utilisation des contrôles UWP XAML C++ Windows dans les applications WinForms ou WPF est un challenge. Techniquement, le Windows Community Toolkit masque la complexité des API COM sous-jacentes.

Videos :

<https://channel9.msdn.com/Shows/Visual-Studio-Toolbox/XAML-Islands>

<https://channel9.msdn.com/Shows/On-NET/Integrating-UWP-components-into-Win32-applications>



reste cadre de 185 mm de large x 50 mm de haut